Lesson 4

15.10.2026

```
public class Task1 {
    public static void main(String[] args) {
        int a = 7;
        int b = 2;
        System.out.println(a / b);
    }
}
```

```
public class Task2 {
    public static void main(String[] args) {
        int a = 7;
        int b = 3;
        System.out.println(a % b);
    }
}
```

```java
public class Task3 {
    public static void main(String[] args) {
        int a = 5;
        a += 3;
        System.out.println(a);
        a--;
        System.out.println(a);
    }
}
```

```java
public class Task4 {
    public static void main(String[] args) {
        int a = 5;
        System.out.println(a++);
        System.out.println(++a);
    }
}
```

```
public class Task5 {
    public static void main(String[] args) {
        double x = 9;
        double y = Math.sqrt(x);
        System.out.println(Math.pow(y, 2));
    }
}
```

Масив – це структура даних, у якій зберігаються елементи одного типу. Його можна уявити, як набір пронумерованих осередків, у кожну з яких можна помістити якісь дані (один елемент даних в одну комірку). Доступ до конкретного осередку здійснюється через її номер. Номер елементу в масиві також називають індексом

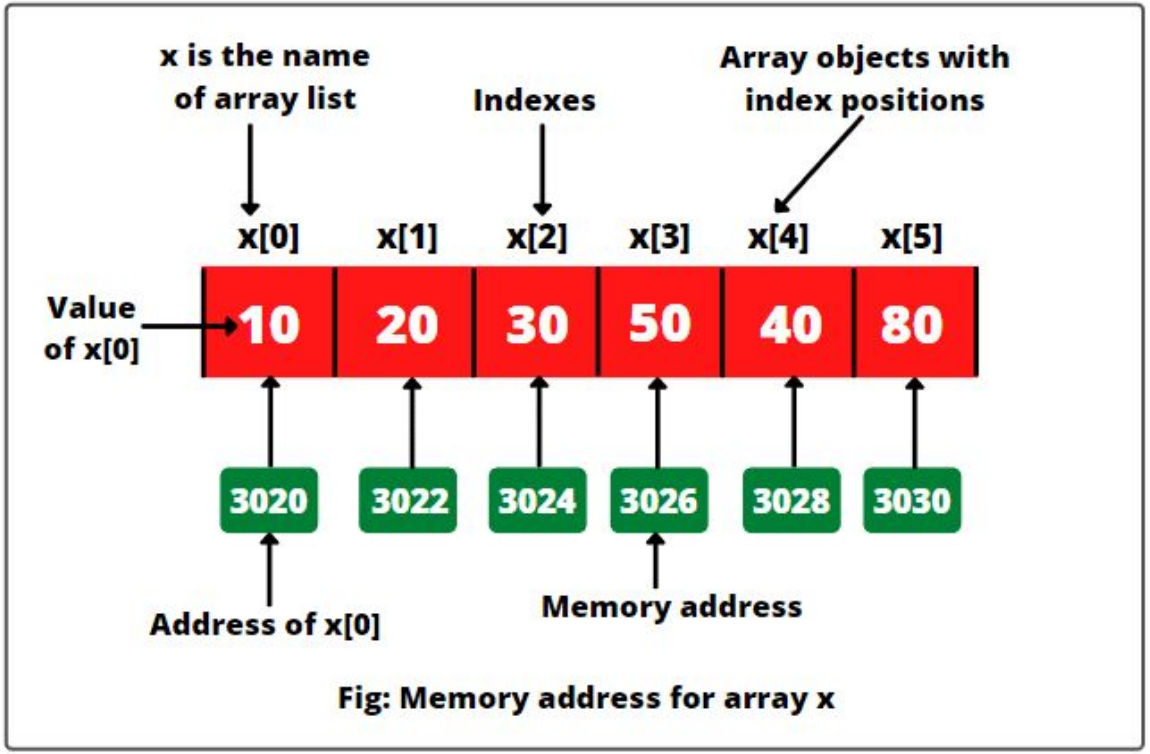
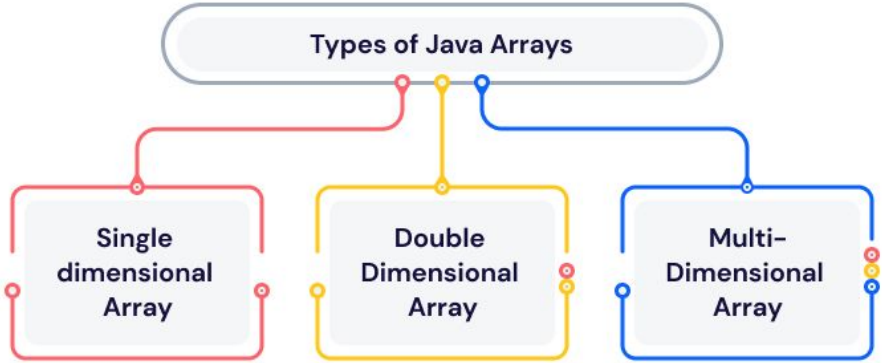


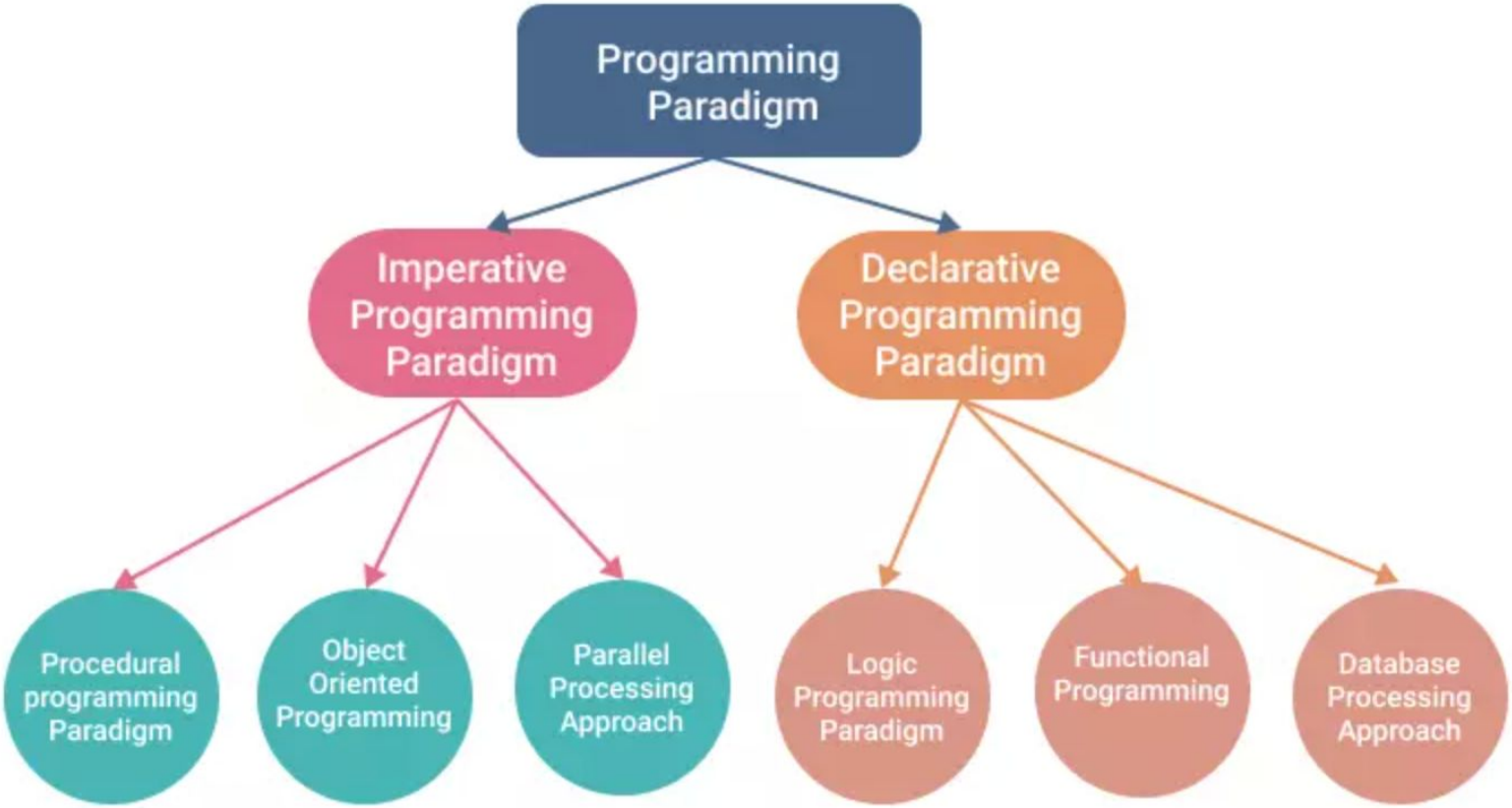
Fig: Memory address for array x

Нижче наведено деякі важливі моменти щодо масивів Java.

- Масиви в Java є об’єктами.
- У Java всі масиви розподіляються динамічно.
- Масив Java також можна використовувати як статичне поле, локальну змінну або параметр методу.
- Розмір масиву має бути задано значенням int, а не long або short.
- Прямим суперкласом типу масиву є Object.
- Масив може містити примітивні типи даних, а також об’єкти класу залежно від визначення масиву.
- У разі примітивних типів даних фактичні значення зберігаються в безперервних розташуваннях пам’яті.
- У випадку об’єктів класу фактичні об’єкти зберігаються в сегменті heap.

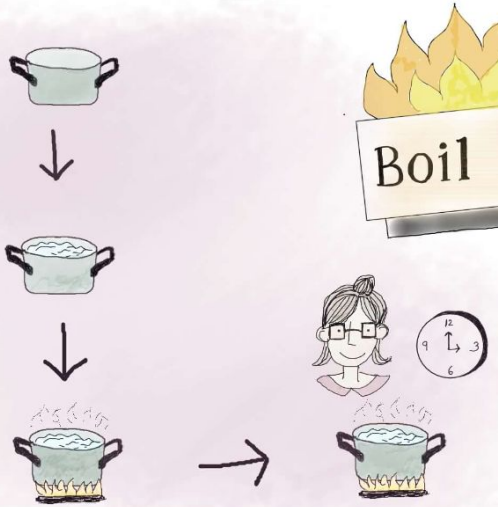


Ознака	Java	C
🔢 **Розмір масиву**	Задається під час створення через `new`, фіксований	Фіксований або динамічний (через `malloc`)
🔍 **Перевірка меж**	Так — `ArrayIndexOutOfBoundsException`	Ні — можливий вихід за межі (UB)
🧱 **Структура**	Масив — це об'єкт у купі	Масив — це послідовна область в пам’яті
🧱 **Типи даних**	Примітиви або об'єкти	Примітиви або структури
🔗 **Арифметика указівників**	Немає	Є — можна пересуватися по масиву
🧹 **Керування пам’яттю**	Автоматичне (GC)	Ручне (`malloc` / `free`)
🧭 **Многовимірність**	Масив масивів (зубчастий)	Справжній багатовимірний масив
🛠️ **Функціональність**	`Arrays.sort`, `stream()` тощо	Немає — все вручну

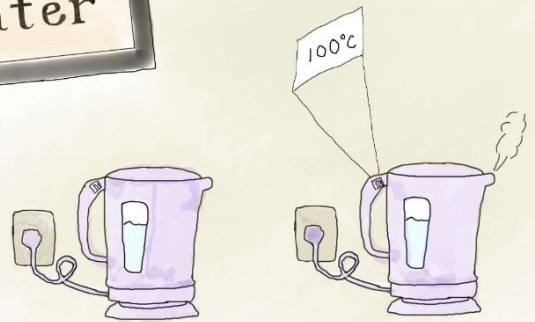

Programming Paradigm
Imperative Programming Paradigm
Declarative Programming Paradigm
Procedural programming Paradigm
Object Oriented Programming
Parallel Processing Approach
Logic Programming Paradigm
Functional Programming
Database Processing Approach

IMPERATIVE VS DECLARATIVE

WHAT DOES IT MEAN?



Boil water



Follow steps

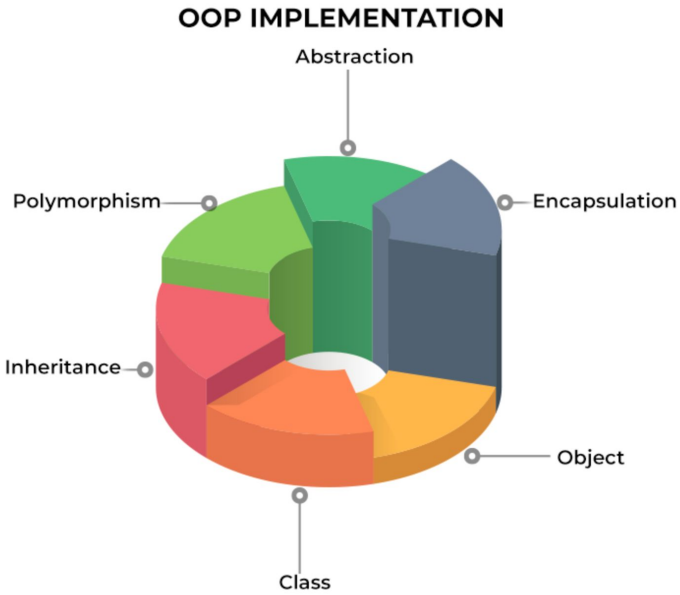
1. Get a pot
2. Pour water in pot
3. Heat pot over stove
4. Wait 5 minutes

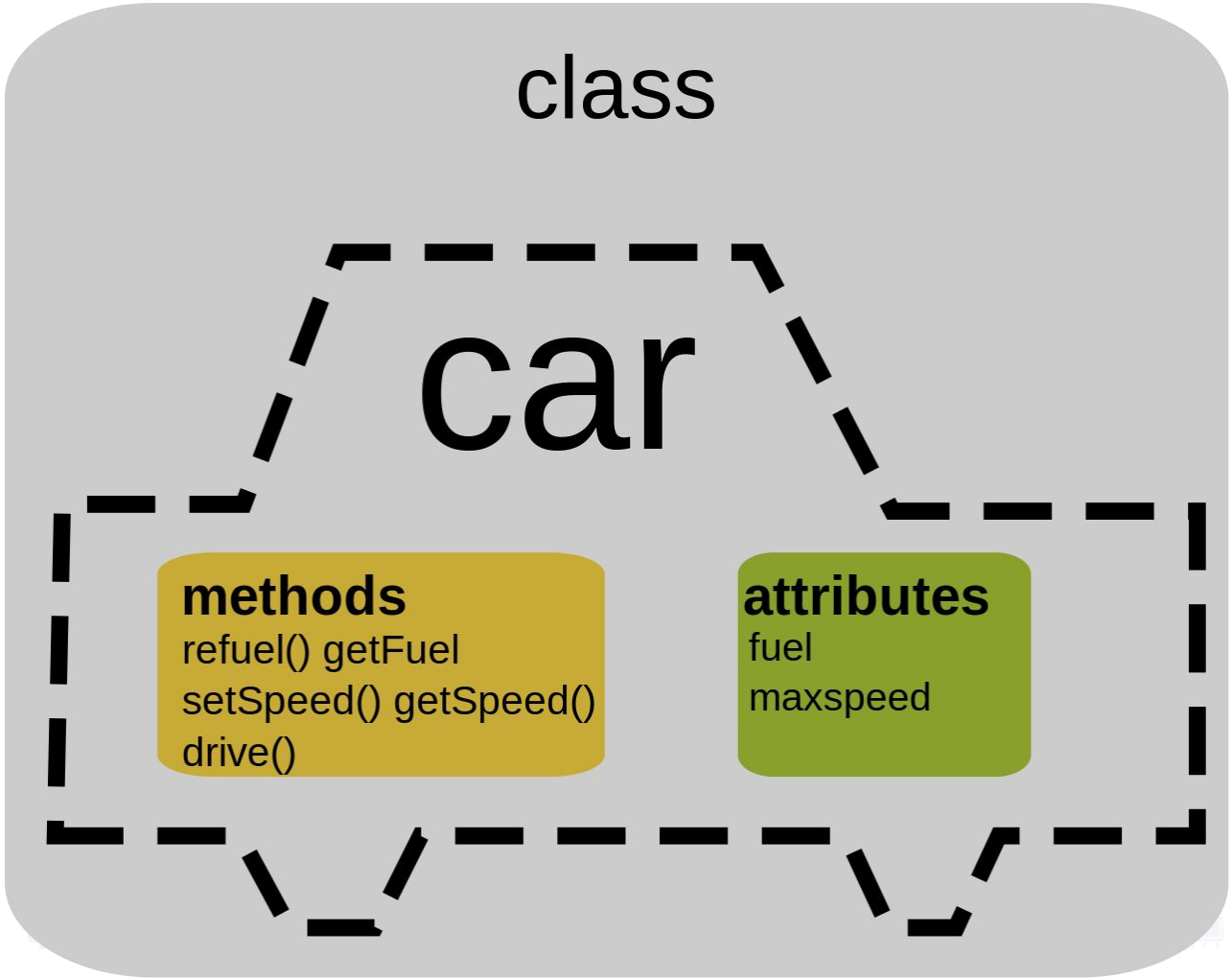
Define what we need

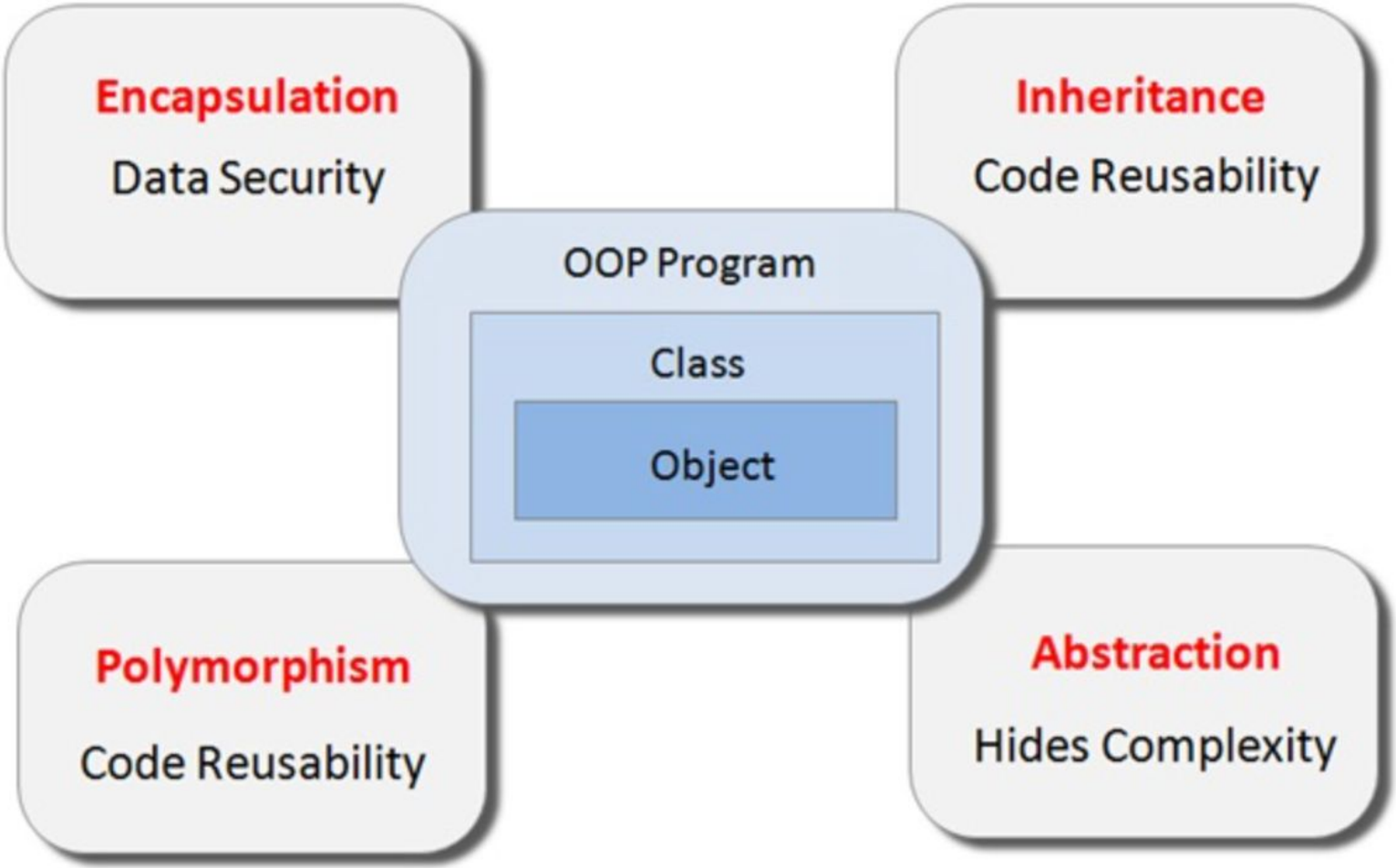
1. Fill kettle so it contains 1L water
2. Heat water in kettle until temperature reaches 100°C

ООР

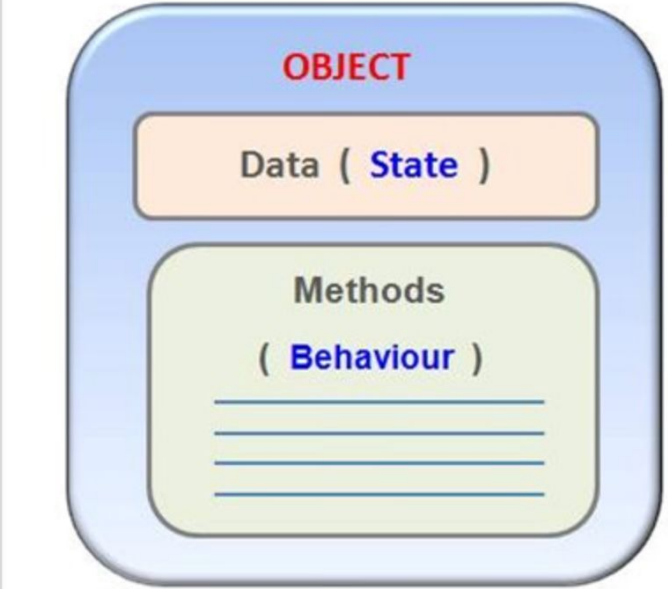
У цій парадигмі програмування програма розбивається на об'єкти – структури даних, що складаються з полів, що описують стан, та методів – підпрограм, що застосовуються до об'єктів для зміни чи запиту їх стану. Більшість об'єктно орієнтованих парадигм для опису об'єктів використовуються класи, об'єкти вищого порядку, що описують структуру та операції, пов'язані з об'єктами



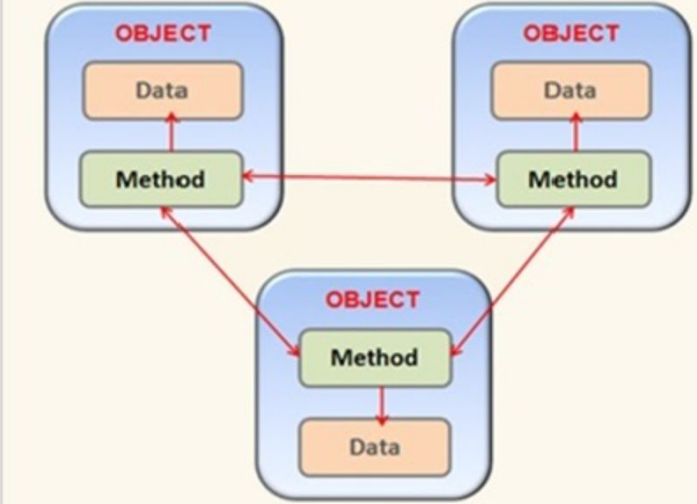

class
car
methods
refuel() getFuel
setSpeed() getSpeed()
drive()
attributes
fuel
maxspeed


Encapsulation
Data Security
Inheritance
Code Reusability
OOP Program
Class
Object
Polymorphism
Code Reusability
Abstraction
Hides Complexity

OOP - Concept Of Object



OPP - Program Organization



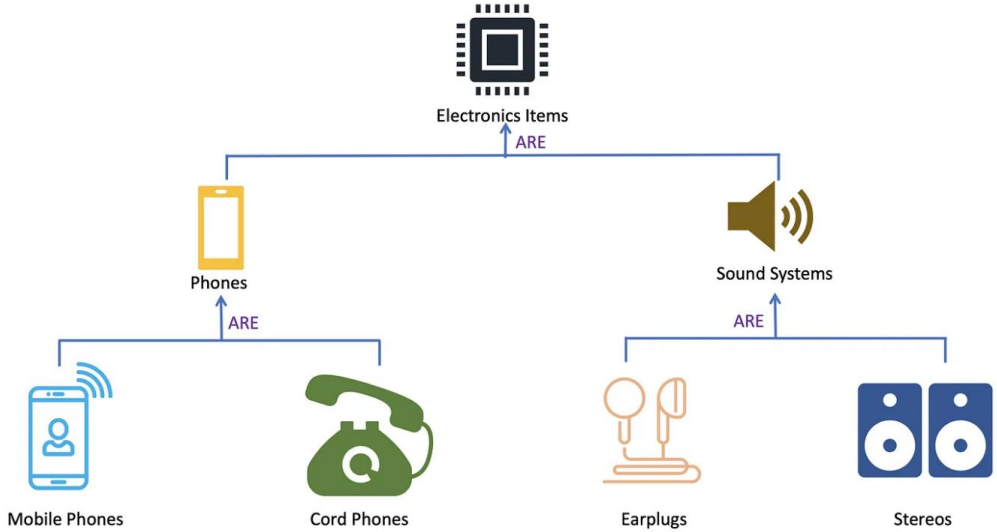
Принципи ООП

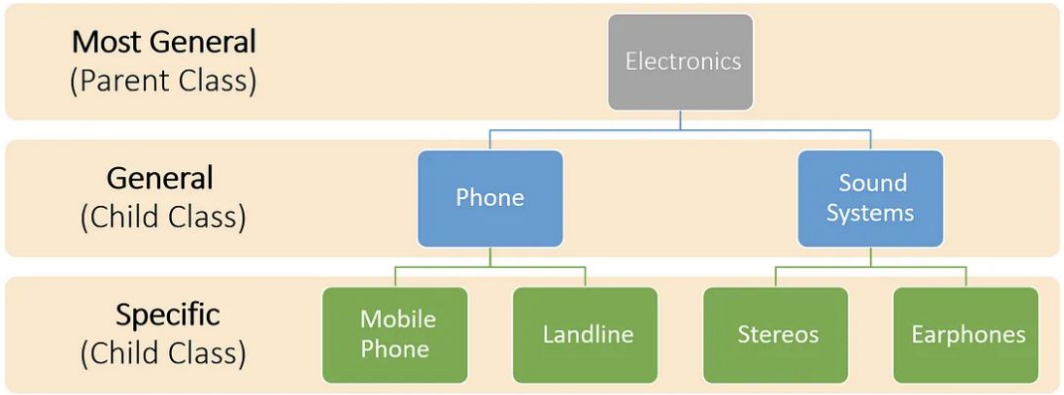
Інкапсуляція це властивість системи, що дозволяє об'єднати дані та методи в класі, та приховати деталі реалізації від користувача.

Наслідування це властивість системи, що дозволяє описати новий клас на основі вже існуючого з частково або повністю запозиченої функціональністю

Поліморфізм – один інтерфейс, безліч методів". Реалізації поліморфізму в мові Java - це навантаження та перевизначення методів, інтерфейси

Абстракція даних це спосіб виділити набір значних показників об'єкта, крім розгляду не значущі. Відповідно, абстракція – це набір всіх таких показників.


Electronics Items
ARE
Phones
ARE
Sound Systems
ARE
Mobile Phones
Cord Phones
Earplugs
Stereos


Most General
(Parent Class)
Electronics
General
(Child Class)
Phone
Sound Systems
Specific
(Child Class)
Mobile Phone
Landline
Stereos
Earphones

ІНКАПСУЛЯЦІЯ

Інкапсуляція — це властивість системи, що дозволяє **об'єднати дані та методи в класі**, та **приховати деталі реалізації** від користувача.



ПЕРЕВАГИ

Захист даних
Запобігає випадковій зміні стану об'єкта.

Контроль доступу
Доступ до даних тільки через визначені методи.

Гнучкість
Можна змінювати реалізацію не впливаючи на користувача.

НАСЛІДУВАННЯ

Наслідування — це властивість системи, що дозволяє описати новий клас на основі вже існуючого з частково або повністю запозиченої функціональністю.

БАЗОВИЙ КЛАС

Тварина



Поля:
- ім'я
- вік
- вага

Методи:
- дихати()
- їсти()
- рухатись()
- спати()

Базовий клас містить спільні поля та методи, які можуть бути використані в похідних класах.

Похідні класи успадковують поля та методи базового класу

ПОХІДНИЙ КЛАС

Кіт



Додатково:
- муркотіти()
- дряпати()

ПОХІДНИЙ КЛАС

Собака



Додатково:
- гавкати()
- бігати()

ПОХІДНИЙ КЛАС

Птах



Додатково:
- літати()
- співати()

Переваги наслідування:
- Повторне використання коду
- Легше розширення системи
- Поліморфізм

Приклад коду (Java):

```
class Тварина {
   String ім'я;
   int вік;
   double вага;

   void дихати() { ... }
   void їсти() { ... }
   void рухатись() { ... }
   void спати() { ... }
}
```

```
class Кіт extends Тварина {

   void муркотти() { ... }
   void дряпати() { ... }
}
```

```
class Собака extends Тварина {

   void гавкати() { ... }
   void бігати() { ... }
}
```

```
class Птах extends Тварина {

   void літати() { ... }
   void співати() { ... }
}
```

Похідні класи можуть перевизначати успадковані методи та додавати нову функціональність.

ПОЛІМОРФІЗМ

Поліморфізм — один інтерфейс, безліч методів.

Реалізації поліморфізму в мові Java - це навантаження та перевизначення методів, інтерфейси.

Суть поліморфізму

Один і той самий виклик методу може мати різну реалізацію залежно від типу об'єкта.

```
Animal a;
a = new Cat();    a.speak(); // Мяу!
a = new Dog();    a.speak(); // Гав!
a = new Bird();   a.speak(); // Чік-чірік!
```

Інтерфейс (тип)

```
public interface Animal {
    void speak();
}
```

Безліч реалізацій



Cat

```
@Override
public void speak() {
    System.out.println("Мяу!");
}
```



Dog

```
@Override
public void speak() {
    System.out.println("Гав!");
}
```



Bird

```
@Override
public void speak() {
    System.out.println("Чік-чірік!");
}
```

РЕАЛІЗАЦІЇ ПОЛІМОРФІЗМУ В JAVA

1. НАВАНТАЖЕННЯ МЕТОДІВ (Method Overloading)

Один клас має декілька методів з однаковим ім'ям, але з різними параметрами (кількість або типи).

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Приклад виклику:

```
Calculator calc = new Calculator();
calc.add(2, 3);        // int
calc.add(2.5, 3.5);    // double
calc.add(1, 2, 3);     // int (3п.)
```

Вибір потрібного методу відбувається на етапі компіляції на основі кількості та типів параметрів.

2. ПЕРЕВИЗНАЧЕННЯ МЕТОДІВ (Method Overriding)

Підклас надає свою реалізацію методу, успадкованого від батьківського класу.

```
class Animal {
    void speak() {
        System.out.println("Тварина видає звук");
    }
}
class Dog extends Animal {
    @Override
    void speak() {
        System.out.println("Гав!");
    }
}
```

Приклад:

```
Animal a = new Dog();
a.speak();
// Гав!
```

Вибір реалізації методу відбувається на етапі виконання програми (динамічний поліморфізм).

3. ІНТЕРФЕЙСИ (Interfaces)

Різні класи реалізують один інтерфейс, надаючи свою реалізацію методів.

```
interface Animal {
    void speak();
}
class Cat implements Animal {
    public void speak() {
        System.out.println("Мяу!");
    }
}
class Bird implements Animal {
    public void speak() {
        System.out.println("Чік-чірік!");
    }
}
```

Приклад:

```
Animal a;
a = new Cat();
a.speak();  // Мяу!

a = new Bird();
a.speak();  // Чік-чірік!
```

Інтерфейс визначає контракт (що робити), а класи надають власну реалізацію (як робити).

Переваги поліморфізму:

- Гнучкість коду
- Розширюваність
- Повторне використання
- Легша підтримка та тестування

АБСТРАКЦІЯ ДАНИХ

Абстракція даних – це спосіб виділити набір **значних показників** об'єкта, ігноруючи **незначущі деталі**.
Відповідно, **абстракція** – це набір всіх таких показників.

РЕАЛЬНИЙ ОБ'ЄКТ

Об'єкт у реальному світі має багато характеристик, але не всі вони важливі для нашої задачі.

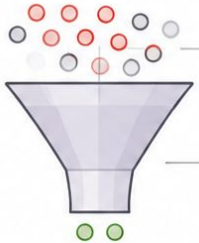


УСІ ХАРАКТЕРИСТИКИ

- Порода
- Колір шерсті
- Кількість шерсті
- Довжина хвоста
- Розмір лап
- Колір очей
- Настрій
- Вік
- Вага
- Улюблена їжа
- … та інші дрібні деталі

ПРОЦЕС АБСТРАКЦІЇ

Ми виділяємо лише ті показники, які є значущими для вирішення задачі.



Незначущі деталі ігноруються

Значущі показники виділяються

АБСТРАКТНА МОДЕЛЬ

Абстрактна модель містить лише значущі показники об'єкта.



АБСТРАКЦІЯ "КІТ"

- Ім'я
- Вік
- Вага
- Настрій

Ці показники достатні для нашої задачі.

ПРИКЛАД У JAVA

Реальний об'єкт (з багатьма полями)

```
class Cat {
   String name;
   String breed;
   String furColor;
   int furLength;
   int tailLength;
   String eyeColor;
   int age;
   double weight;
   String mood;
   String favoriteFood;
   // ... інші деталі
}
```

Абстракція – виділяємо тільки значущі поля

Абстрактна модель (лише значущі поля)

```
class Cat {
   private String name;
   private int age;
   private double weight;
   private String mood;

   // Доступ до даних через методи

   public String getName() { ... }
   public int getAge() { ... }
   public double getWeight() { ... }
   public String getMood() { ... }
}
```

ПЕРЕВАГИ АБСТРАКЦІЇ

Фокус на головному
Ми працюємо лише з тим, що важливо для вирішення задачі.

Спрощення складності
Приховуємо зайві деталі, зменшуємо складність системи.

Легке обслуговування
Зміни в незначущих деталях не впливають на інші частини програми.

Зрозумілий інтерфейс
Користувач взаємодіє з простим та зрозумілим інтерфейсом.

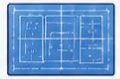
Class vs. Object

ОБ’ЄКТ vs КЛАС

Основні відмінності в ООП





ОБ’ЄКТ		КЛАС
Об’єкт – це екземпляр класу. Об’єкт створюється на основі класу. 	Визначення	 Клас – це «шаблон» (опис), за яким створюються об’єкти.
Об’єкт – це реальна сутність світу: наприклад, стілець, ручка, стіл, ноутбук тощо. 	Природа	 Клас – це група схожих об’єктів. Описує їхні спільні властивості та поведінку.
Об’єкт – це фізична сутність. 	Тип сутності	</> Клас – це логічна сутність (опис у програмі).
 Об’єктів можна створювати багато разів за потреби. 	Створення	CLASS Клас оголошується (визначається) один раз.
 Об’єкт займає пам’ять при створенні. 	Пам’ять	 Клас не займає пам’ять при оголошенні.
 Об’єкт створюється за допомогою ключового слова **new**. Наприклад: `Cat cat = new Cat();` 	Створення в коді	</>  Клас оголошується за допомогою ключового слова **class**. Наприклад: `class Cat { }`
 Існує декілька способів створити об’єкт у Java: • за допомогою **new** • за допомогою методів (newInstance()) • за допомогою **clone()** • шляхом десеріалізації 	Способи створення	 Існує лише один спосіб визначення класу – за допомогою ключового слова **class**.



Підсумок:
Клас – це план (опис), а об’єкт – це конкретний екземпляр, створений за цим планом і який має власний стан та поведінку.



Приклад на котиках:
Клас Cat – це опис усіх котиків.
Об’єкт cat1, cat2, cat3 – це конкретні котики, кожен зі своїм іменем, віком, вагою тощо.





Breed: Bulldog
Size: large
Colour: light gray
Age: 5 years

Dog1Object



Breed: Beagle
Size: large
Colour: orange
Age: 6 years

Dog2Object



Breed: German Shepherd
Size: large
Colour: white & orange
Age: 6 years

Dog3Object

Dog

Fields
Breed
Size
Colour
Age

Methods
Eat()
Run()
Sleep()
Name()

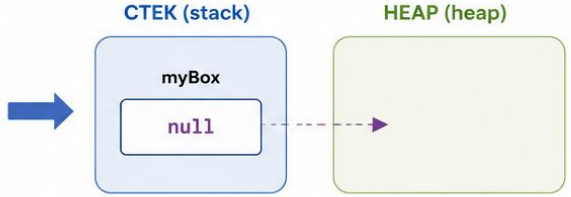
ПАМ'ЯТЬ У JAVA: СТЕК ТА HEAP

У Java об'єкти (посилання) зберігаються в **стеку**, а самі об'єкти – у **heap (кучі)**.

1. ОГОЛОШЕННЯ ПОСИЛАННЯ ТА ПРИСВОЄННЯ null

```
Box myBox;
// оголошення посилання
```

```
myBox = null;
// посилання не вказує
   на жоден об'єкт
```



Що відбувається?

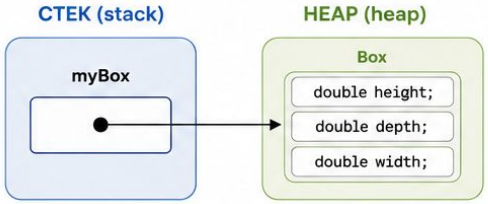
Створюється посилання **myBox** у стеку.
Воно не вказує на жоден об'єкт (null).
В heap нічого не створюється.

Важливо:

null означає відсутність посилання на об'єкт.

2. СТВОРЕННЯ ОБ'ЄКТА ЗА ДОПОМОГОЮ new

```
myBox = new Box();
// створення об'єкта Box
   та збереження посилання
   на нього в myBox
```



Що відбувається?

У heap створюється об'єкт класу Box з трьома полями.
Посилання **myBox** у стеку тепер вказує на цей об'єкт.

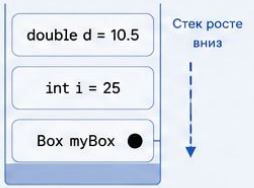
Важливо:

У стеку зберігається тільки посилання, а не сам об'єкт!

СТЕК (stack)

Зберігає:

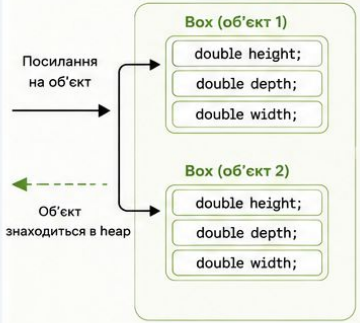
- примітивні типи (int, double, boolean тощо)
- посилання на об'єкти
- локальні змінні методів



Особливості:

Швидкий доступ.
Пам'ять автоматично звільняється, коли метод завершує роботу.

HEAP (heap)



Зберігає:

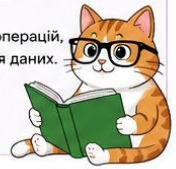
- усі створені об'єкти (за допомогою new)
- масиви
- рядки (String)
- інші складні структури

Особливості:

Більший обсяг, ніж стек.
Пам'ять звільняється Garbage Collector (GC) автоматично.

НА ЩО ЗВЕРНУТИ УВАГУ

- Посилання – це змінна, яка містить адресу об'єкта в heap.
- Коли посилання = null – об'єкт не доступний (якщо немає інших посилань) і буде видалений GC.
- Heap живе довше, ніж стек.
- Стек для швидких операцій, heap для зберігання даних.



У мові Java під час проектування класів прийнято обмежувати рівень доступу до змінним за допомогою модифікатора **private** і звертатися до них через гетери та сетери.

Існують правила оголошення таких методів, розглянемо їх:

- Якщо властивість НЕ типу boolean, префікс геттера має бути get. Наприклад: **getName()** - це коректне ім'я геттера для змінної name.
- Якщо властивість типу boolean, префікс імені геттера може бути get або is. Наприклад, **getPrinted()** або **isPrinted()** обидва є коректними іменами для змінних типу boolean.
- Ім'я сеттера повинне починатися з префікса set. Наприклад, **setName()** коректне ім'я для змінної name.
- Для створення імені геттера або сеттера, перша літера властивості має бути змінена на велику та додана до відповідного префіксу (set, get або is).
- Сеттер має бути public, повертати void тип і мати параметр відповідний типу змінної.
- Геттер метод повинен бути public, не мати параметрів методу, і повертати значення відповідне типу властивості

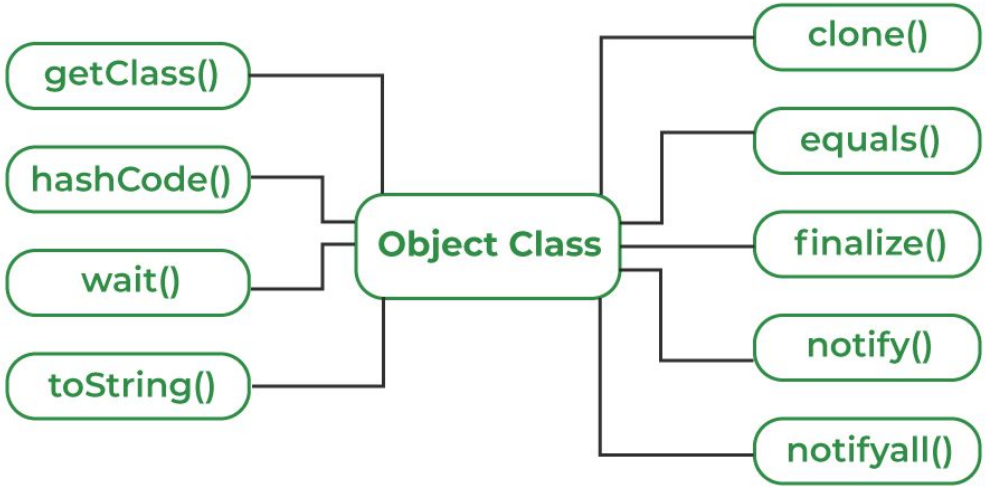
Клас Object

Є суперкласом для всіх класів (включаючи масиви)

Змінна цього типу може посилатися будь-який об'єкт (але не змінну примітивного типу)

Його методи успадковуються усіма класами

Реалізує базові операції з об'єктами



Method	Purpose
Object clone( )	Creates a new object that is the same as the object being cloned.
boolean equals(Object *object*)	Determines whether one object is equal to another.
void finalize( )	Called before an unused object is recycled.
Class<?> getClass( )	Obtains the class of an object at run time.
int hashCode( )	Returns the hash code associated with the invoking object.
void notify( )	Resumes execution of a thread waiting on the invoking object.
void notifyAll( )	Resumes execution of all threads waiting on the invoking object.
String toString( )	Returns a string that describes the object.
void wait( ) void wait(long *milliseconds*) void wait(long *milliseconds*, int *nanoseconds*)	Waits on another thread of execution.

Функція – частина програми, яка має власне ім'я. Це ім'я можна використовувати у програмі як команду (така команда називається викликом функції). Під час виклику функції виконуються команди, з яких вона складається. Виклик функції може повертати значення (аналогічно операції) і тому може використовуватися у виразі поряд з операціями.

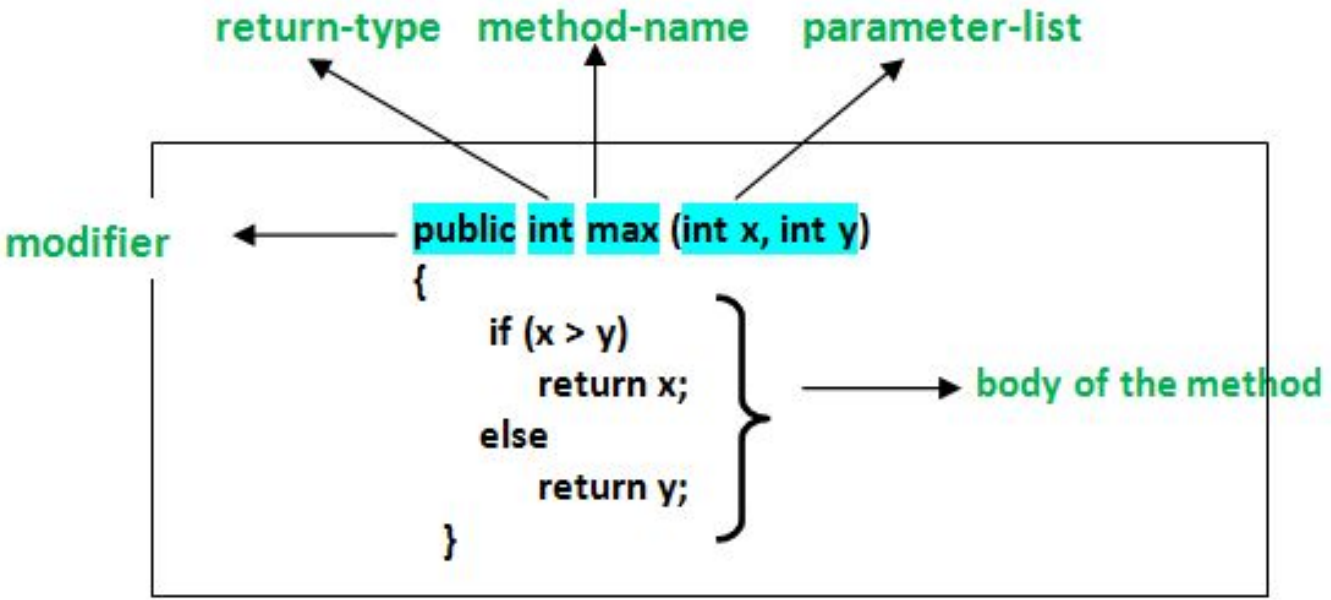
Метод - це функція, яка є частиною деякого класу, яка може виконувати операції над даними цього. У мові Java вся програма складається тільки з класів і функції можуть описуватись тільки всередині них. Саме тому всі функції мови Java є методами.

Функції використовуються у програмуванні, щоб зменшити його складність:

1.Замість того, щоб писати безперервну послідовність команд, якої незабаром перестаєш орієнтуватися, програму розбивають на підпрограми, кожна з яких вирішує невелике закінчене завдання, а потім велика програма складається з цих підпрограм (цей прийом називається декомпозицією).

2.Зменшується загальна кількість коду, тому що, як правило, одна функція використовується у програмі кілька разів.

3.Написана якось і всебічно перевірена функція, можливо включена до бібліотеки функцій та використовуватись в інших програмах (при цьому не треба згадувати, як була запрограмована ця функція, достатньо знати, що вона робить).


return-type
method-name
parameter-list
modifier
public int max (int x, int y)
{
if (x > y)
return x;
else
return y;
}
body of the method

У мові Java під час проектування класів прийнято обмежувати рівень доступу до змінним за допомогою модифікатора **private** і звертатися до них через гетери та сетери.

Існують правила оголошення таких методів, розглянемо їх:

- Якщо властивість НЕ типу boolean, префікс геттера має бути get. Наприклад: **getName()** - це коректне ім'я геттера для змінної name.
- Якщо властивість типу boolean, префікс імені геттера може бути get або is. Наприклад, **getPrinted()** або **isPrinted()** обидва є коректними іменами для змінних типу boolean.
- Ім'я сеттера повинне починатися з префікса set. Наприклад, **setName()** коректне ім'я для змінної name.
- Для створення імені геттера або сеттера, перша літера властивості має бути змінена на велику та додана до відповідного префіксу (set, get або is).
- Сеттер має бути public, повертати void тип і мати параметр відповідний типу змінної.
- Геттер метод повинен бути public, не мати параметрів методу, і повертати значення відповідне типу властивості

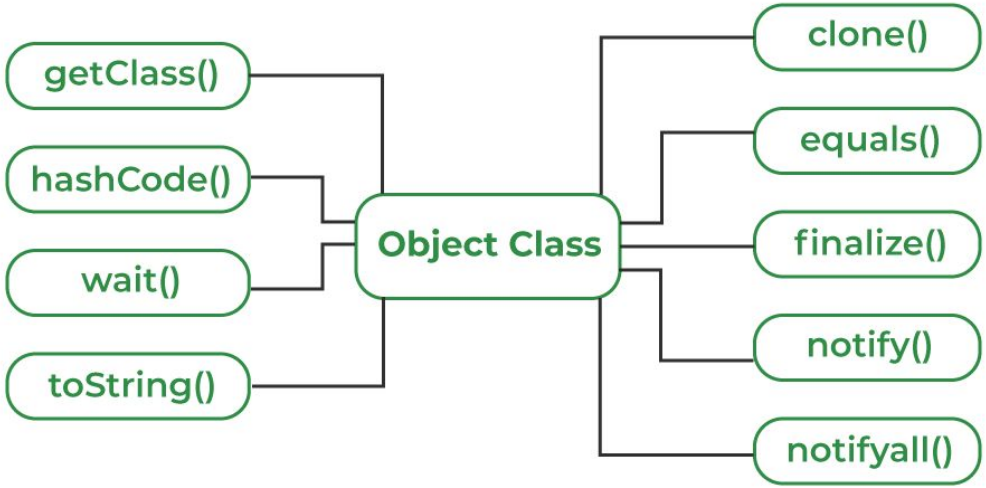
Клас Object

Є суперкласом для всіх класів (включаючи масиви)

Змінна цього типу може посилатися будь-який об'єкт (але не змінну примітивного типу)

Його методи успадковуються усіма класами

Реалізує базові операції з об'єктами





КЛАС Object У JAVA

Клас Object є батьківським для всіх класів у Java.
Він містить базові методи, доступні для кожного об'єкта.

Усі класи в Java неявно успадковують методи класу Object.

getClass()

Повертає об'єкт класу Class, що відповідає класу цього об'єкта.

Повертає: Class<?>

```
obj.getClass(); // class java.lang.String
```

hashCode()

Повертає хеш-код (ціле число) для даного об'єкта.

Повертає: int

```
int code = obj.hashCode(); // 12345678
```

wait()

Призупиняє виконання потоку до отримання повідомлення (notify).

Повертає: void

```
obj.wait();
```

toString()

Повертає текстове представлення об'єкта.

Повертає: String

```
String s = obj.toString(); // клас@hashcode
```

Примітка:
Більшість методів класу Object рідко перевизначаються, але деякі з них (toString(), equals(), hashCode()) часто перевизначаються у власних класах.

Object Class

clone()

Створює та повертає копію об'єкта.

Повертає: Object

```
Object copy = obj.clone();
```

equals(Object obj)

Порівнює даний об'єкт з іншим на рівність.

Повертає: boolean

```
boolean isEqual = obj.equals(other);
```

finalize()

Викликається перед тим, як об'єкт буде знищений збирачем сміття.

Повертає: void

```
protected void finalize() { ... }
```

notify()

Пробуджує один потік, який очікує на цьому об'єкті.

Повертає: void

```
obj.notify();
```

notifyAll()

Пробуджує всі потоки, які очікують на цьому об'єкті.

Повертає: void

```
obj.notifyAll();
```

КОНСТРУКТОР У JAVA



Конструктор – це метод, призначення якого полягає у створенні екземпляра класу (об'єкта).

JVM

Виклик

Конструктори в Java

За замовчуванням (неявний, від Java)

Якщо в класі не оголошено жодного конструктора, Java автоматично додає конструктор без параметрів.



Приклад:

```
class MyClass {
   public MyClass() {
     System.out.println("Об'єкт створено");
   }
}
MyClass obj = new MyClass();
```

Виклик

Без параметрів (явний)

Конструктор без параметрів, який явно оголошений у класі.



З параметрами (явний)

Конструктор з параметрами, який явно оголошений у класі.



Виклик

Приклад:

```
class MyClass {
private int id;
private String name;

public MyClass(int id, String name) {
    this.id = id;
    this.name = name;
    System.out.println("Об'єкт створено:
      " + id + ", " + name);
}
}
MyClass obj = new MyClass(1, "Іван");
```

Підсумок:

Конструктори викликаються при створенні об'єкта за допомогою оператора **new**.
Вони можуть бути неявними (за замовчуванням) або явними (з параметрами або без них).

Перевантаження функцій
(Function Overloading) в програмуванні

Один і той самий метод може мати кілька версій у межах одного класу з різними параметрами. Компілятор визначає, який метод викликати, залежно від кількості та типів аргументів.

Приклад

```
class MyClass {
    int my_sum(int a, int b) {
        return a + b;
    }

    int my_sum(int a, int b, int c) {
        return a + b + c;
    }
}
```

Діаграма класу

MyClass

+ my_sum(a: int, b: int) : int
+ my_sum(a: int, b: int, c: int) : int

Ключова ідея:

Методи мають однакове ім'я, але різні списки параметрів (кількість або типи). Повернення типу може бути тим самим або іншим.

Перевизначення функцій
(Function Overriding) в програмуванні

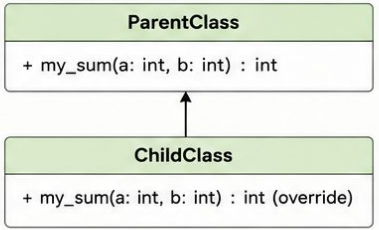
Дочірній клас надає свою реалізацію методу, який уже існує в батьківському класі з таким самим ім'ям, параметрами та типом повернення. Виклик методу залежить від об'єкта.

Приклад

```
class ParentClass {
    int my_sum(int a, int b) {
        return a + b;
    }
}

class ChildClass extends ParentClass {
    @Override
    int my_sum(int a, int b) {
        return a - b;  // перевизначена реалізація
    }
}
```

Діаграма класів

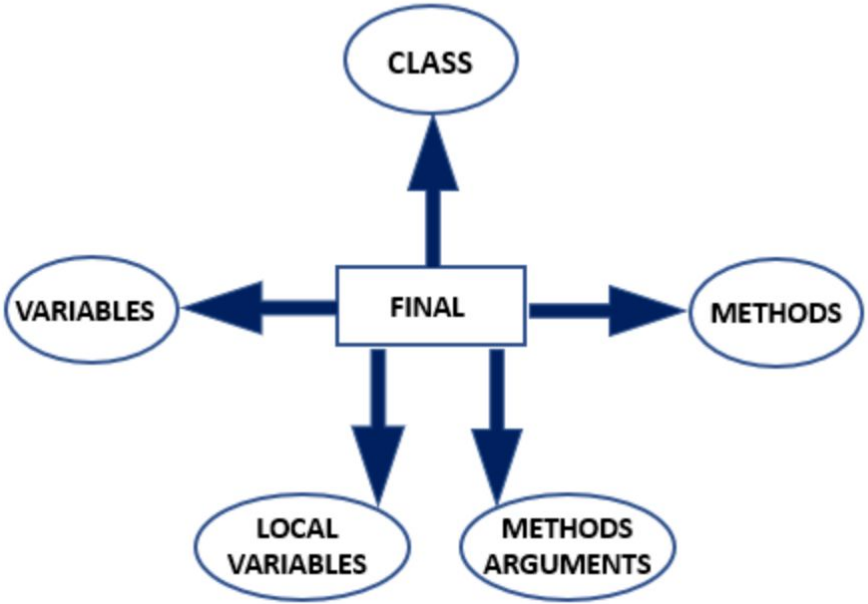


Ключова ідея:

Метод у дочірньому класі має ту саму сигнатуру (ім'я, параметри, тип повернення), що й у батьківському класі.

Access Modifiers	Non-Access Modifiers
private default or No Modifier protected public	static final abstract synchronized transient volatile strictfp

1. Final Non Access Modifiers



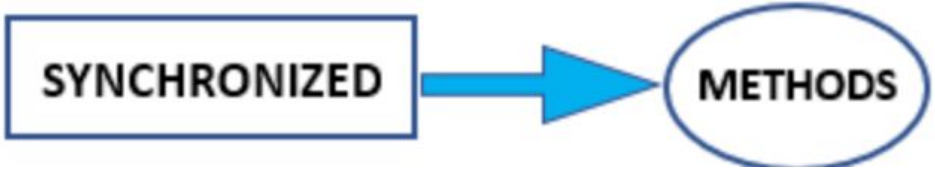
Final ключове слово використовується з класом, коли ми хочемо обмежити його успадкування будь-яким іншим класом. Наприклад, якщо у нас є final клас, то будь-яка спроба розширити цей клас може призвести до помилки під час компіляції.

2. Abstract Non-Access Modifier



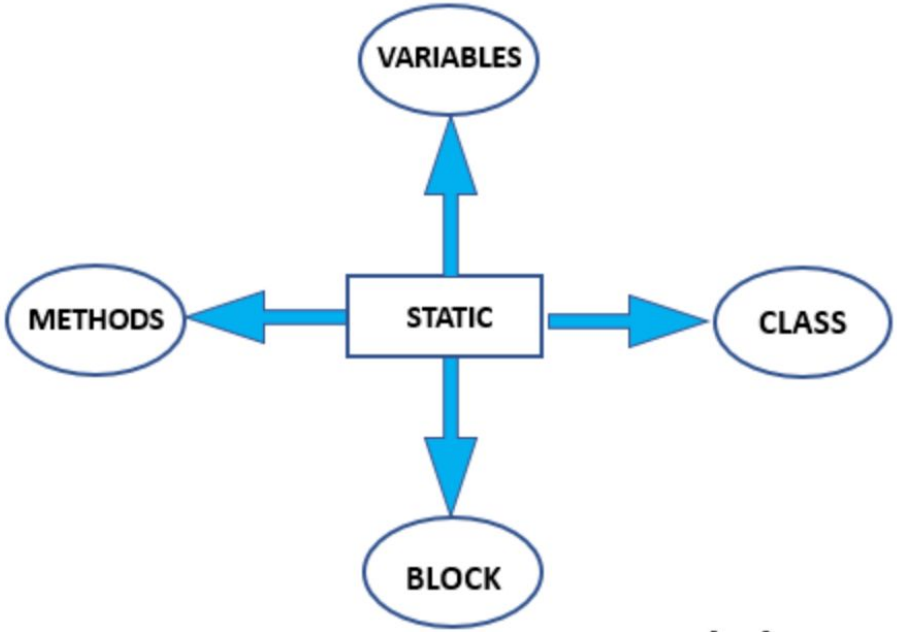
Клас оголошується як абстрактний, щоб вказати, що цей клас не може бути створений, що означає, що жодні об’єкти не можуть бути сформовані для цього класу, але можуть бути успадковані. Тим не менш, цей клас має конструктор, який буде викликатися всередині конструктора його підкласу. Він може містити як абстрактні, так і фінальні методи, де абстрактні методи будуть замінені в підкласі.

3. Synchronized Non-Access Modifier



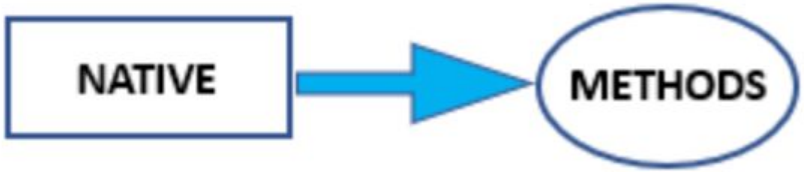
Це ключове слово допомагає запобігти одночасному доступу кількох потоків до одного методу, таким чином синхронізуючи потік програми та виводячи бажані результати за допомогою функції багатопоточності.

4. Static Non-Access Modifier



Ця змінна використовується для керування пам’яттю та для першого посилання під час завантаження класу. До цих членів ставляться на рівні класу; таким чином, їх не можна викликати за допомогою об'єкта; натомість ім'я класу використовується для посилання на них.

5. Native Non Access Modifier



Ключове слово native використовується лише з методами, щоб вказати, що конкретний метод написаний залежно від платформи. Вони використовуються для покращення продуктивності системи, а існуючий застарілий код можна легко використовувати повторно.

6. Strictfp Non-Access Modifier



Це ключове слово використовується для того, щоб результати операції над числами з плаваючою комою виводили однакові результати на кожній платформі. Це ключове слово не можна використовувати з абстрактними методами, змінними або конструкторами, оскільки вони не повинні містити операції.

7. Transient Non-Access Modifier

Під час передачі даних від одного кінця до іншого через мережу, вони повинні бути серіалізовані для успішного отримання даних, що означає перетворення в потік байтів перед надсиланням і перетворення його назад на кінці прийому. Щоб повідомити JVM про учасників, які не потребують серіалізації замість того, щоб бути втраченими під час передачі, з'являється тимчасовий модифікатор.

- Ініціалізація статичних полів та блоків виконується під час завантаження класу

- Ініціалізація нестатичних елементів виконується під час створення об'єкта(при виклику конструктора)